

LES AGENTS IA

Architecture · Mind · Body · Memory · ReAct

Un agent = Mind (raisonne) + Body (agit) + Memory (se souvient)
Le LLM n'est qu'une implémentation possible du Mind

**Si (1) le script agit mais ne réfléchit pas (règles), et
que (2) le LLM réfléchit mais n'agit pas ...
A quel moment ce système devient-il un Agent ?**

Un LLM $\neq$ Un agent

Où se situe l'agent IA ?

Un agent n'est pas une discipline — c'est une architecture



Le LLM est un (parmi d'autres) Mind possible — pas une obligation

Agent IA

Un agent est une **entité autonome** capable de **prendre des décisions** tout seul afin **d'atteindre un but** spécifique.
Contrairement à un modèle d'intelligence artificielle classique, l'agent ne se contente pas de répondre à une question ; il interagit avec son **environnement** pour accomplir une mission.

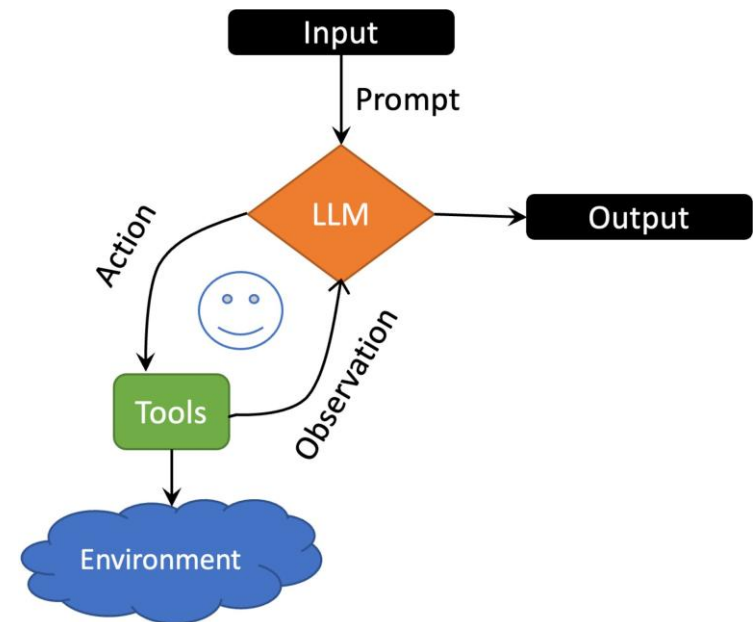
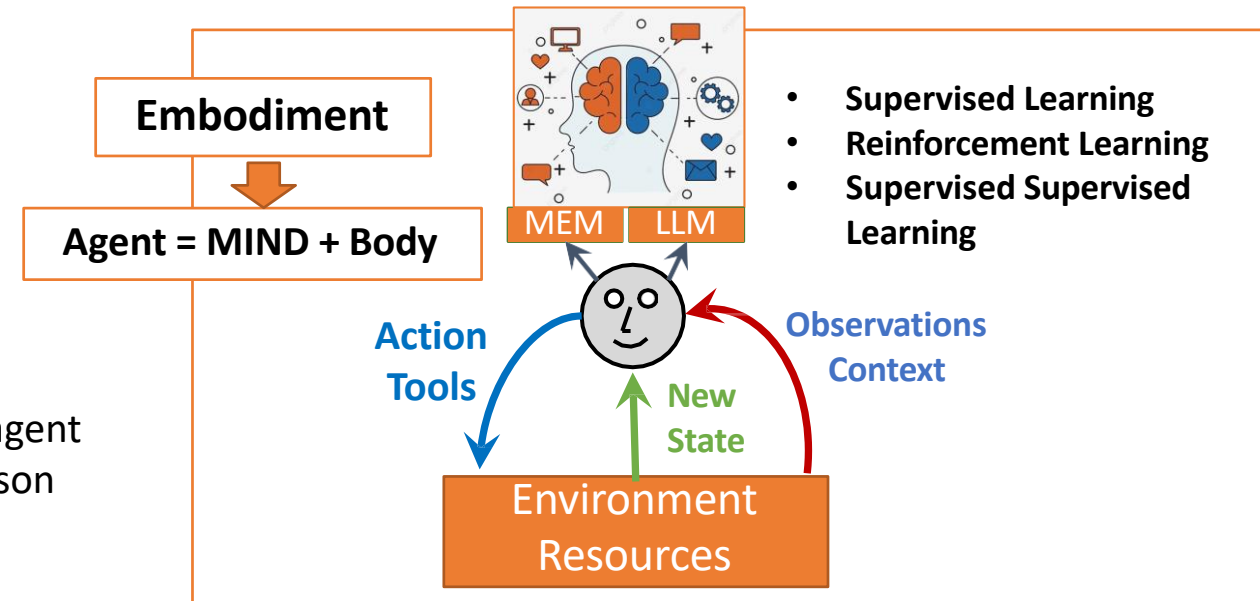
Mind + Body deux dimensions complémentaires :

Cerveau (Mind) : représente la dimension **épistémologique**, c'est-à-dire le raisonnement et la réflexion.

Dans l'IA agentique moderne, ce rôle est assuré par les LLMs, qui permettent à l'agent de comprendre le langage humain et de planifier des actions.

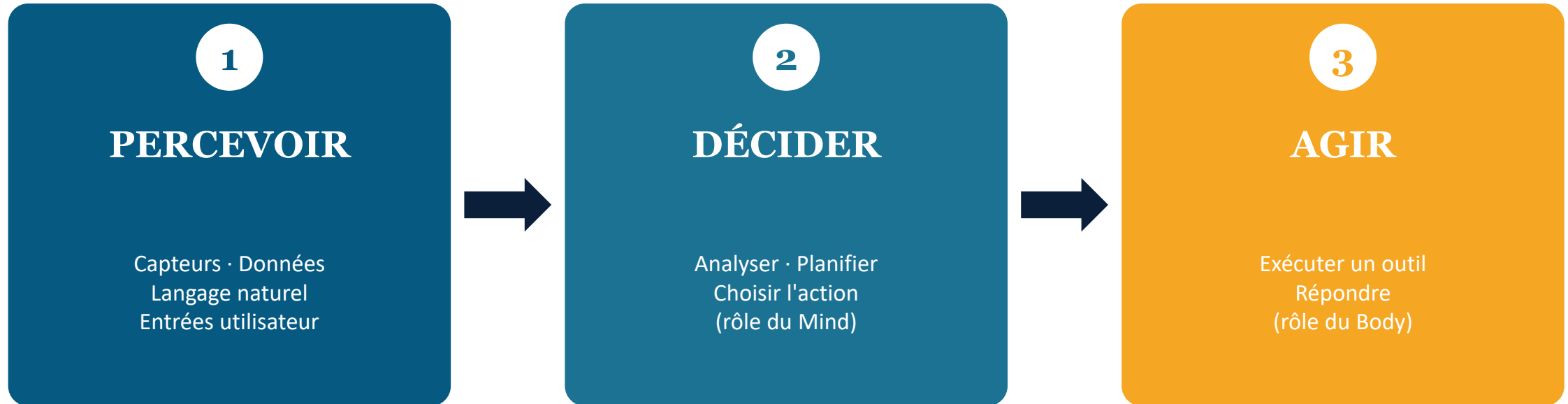
Le corps (Body) : correspond à la dimension **physicaliste**. Il est constitué d'un ensemble d'outils (**tools**) et de fonctions qui permettent à l'agent d'agir sur son environnement.

Ex. Tools: accès à des bases de données, des capteurs IoT, ou des logiciels de productivité comme Google Sheets et Gmail.



Cycle fondamental : Percevoir → Décider → Agir

« Une entité qui perçoit, décide, et agit pour atteindre un objectif »



🔄 Boucle de rétroaction avec l'environnement — Ex. Thermostat : perçoit la T° → compare au seuil → allume le chauffage

Architecture complète d'un agent

Agent = Mind + Body + Memory

MIND

Mécanisme de décision

- Règles (si...alors)
- ML supervisé
- Reinforcement Learning
- LLM ★ cas courant
- Architecture hybride

BODY

Outils d'action

- Bases SQL / NoSQL
- APIs REST & Web
- Exécuteur Python
- Email · Agenda · Slack
- RAG / Vector DB

MEMORY

Mémoire de l'agent

- Court terme (contexte)
- Long terme (vector DB)
- Épisodique (historique)
- Sémantique (faits)
- Procédurale (skills)

Le Mind raisonne → génère du JSON → le Body exécute → le résultat alimente la Memory → le Mind continue

La Memory — pilier oublié de l'agent

Sans mémoire, un agent est amnésique à chaque requête — 5 types à distinguer

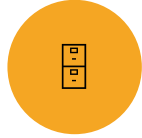


Court terme

Le contexte actif pendant une requête. Limité par la fenêtre de tokens du LLM. Perdu à la fin de la session.

Ex :

derniers messages, variables temporaires



Long terme

Stockage persistant entre sessions. Typiquement une base vectorielle (embeddings) interrogée par similarité.

Ex :

Pinecone, Chroma, Weaviate



Épisodique

Historique des interactions passées de l'agent — ses « expériences » datées et contextualisées.

Ex :

logs de conversations, traces ReAct



Sémantique

Connaissances factuelles sur le monde, indépendantes du contexte. Le « quoi ».

Ex :

knowledge base, wiki interne, RAG



Procédurale

Savoir-faire réutilisables : comment exécuter une tâche. Le « comment ».

Ex :

skills, prompts templates, workflows

Le Mind : 6 implémentations possibles

Comment l'agent decide ? — du plus simple (règles codées) au plus complexe (hybride)

Règles

Définition :

Conditions si...alors écrites à la main par un humain

Exemple :

Thermostat : Si température < 18°C ALORS allumer chauffage

✓ Transparent · zéro donnée

Recherche

Définition :

Explore un arbre d'états pour trouver le meilleur chemin

Exemple :

GPS : teste toutes les routes et choisit la plus courte (A)*

✓ Plan optimal garanti

ML Supervisé

Définition :

Apprend une fonction entrée → sortie à partir d'exemples étiquetés

Exemple :

Détection fraude : entraîné sur 1M transactions « fraude / OK »

✓ Apprend des données

RL (Renforcement)

Définition :

Apprend par essai-erreur avec des récompenses/pénalités

Exemple :

AlphaGo : joue des millions de parties et optimise sa stratégie

✓ S'améliore seul

LLM ★

Définition :

Modèle de langage pré-entraîné qui raisonne en générant du texte

Exemple :

ChatGPT : prédit le mot suivant → produit raisonnement + JSON d'action

✓ Polyvalent · zero-shot

Hybride

Définition :

Combine plusieurs approches selon la tâche à résoudre

Exemple :

Agent SAV : LLM comprend → RAG cherche docs → règles valident

✓ Performance maximale

3 concepts à ne pas confondre

Comportement ≠ mécanisme de décision ≠ méthode d'apprentissage

Types d'AGENT

Le comportement observable

- **Réactif** — répond uniquement aux stimuli, sans plan (thermostat).
- **Proactif** — anticipe et agit sans qu'on le sollicite (alerte météo).
- **Délibératif** — planifie plusieurs étapes avant d'agir (GPS multi-étapes).
- **Hybride** — combine réactif + délibératif selon la situation.

Types de MIND

Le mécanisme de décision

- **Règles** — logique si...alors codée à la main.
- **ML supervisé** — modèle entraîné sur données étiquetées.
- **RL** — apprentissage par récompense.
- **LLM** — modèle de langage génératif.
- **Hybride** — plusieurs mécanismes combinés.

APPRENTISSAGE

Comment le Mind apprend

- **Supervisé** — appris avec couples (entrée, bonne réponse).
- **Non supervisé** — trouve des patterns sans étiquettes.
- **Renforcement** — apprend via récompenses/pénalités.
- **Hybride** — combinaison des trois.

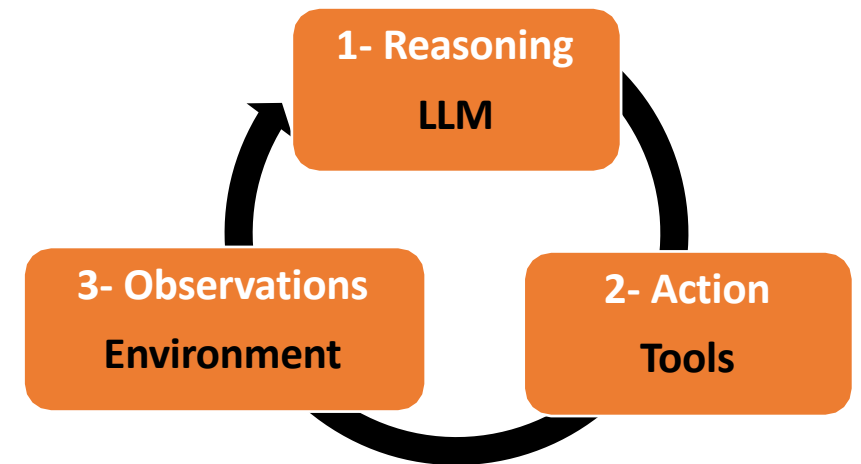
Le Pattern ReAct : Thought → Action → Observation

Pattern de CONTRÔLE — pas un paradigme d'apprentissage



Fonctionnement :

- Un agent opère généralement selon un design pattern nommé React (Reasoning and Action). Ce cycle comprend plusieurs étapes :
 - **Raisonnement** : Le cerveau (LLM) analyse la situation et détermine l'action optimale à prendre.
 - **Action** : L'agent utilise un outil spécifique pour interagir avec l'extérieur.
 - **Observation** : L'agent collecte les résultats de son action (feedback ou contexte). Il renvoie ces observations à son cerveau pour raisonner à nouveau, jusqu'à ce qu'il dispose de tout le contexte nécessaire pour atteindre son but.



Le Pattern ReAct : Thought → Action → Observation

Pattern de CONTRÔLE — pas un paradigme d'apprentissage



Exemple — « 5% du PIB du Maroc 2023 en dirhams ? »

T1 Thought : chercher PIB Maroc 2023 via Wikipedia

Action : `search_wikipedia({query: 'PIB Maroc 2023'})` →

Obs : 141 Mds USD

T2 Thought : calculer 5% puis convertir en MAD

Action : `calculator + convert_currency`

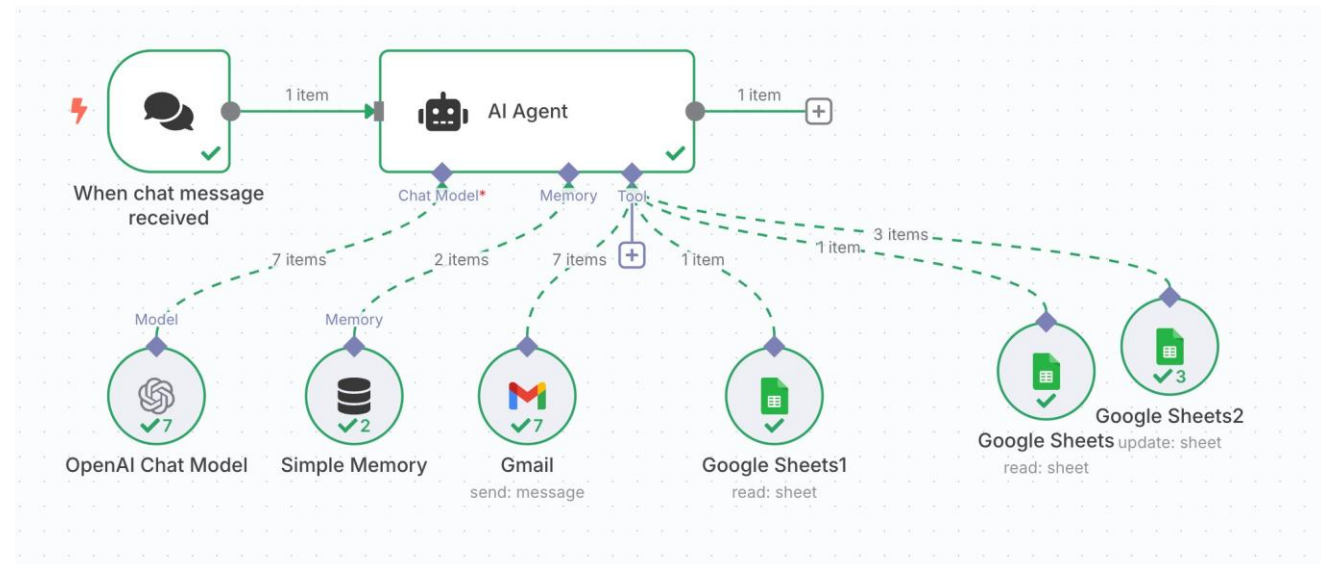
Obs : 70.76 Mds MAD

✓ **Réponse** : 5% ≈ 7.05 Mds USD ≈ 70.76 Mds MAD [3 tours · 0 hallucination]

Demo 1 : First Agent in Automation platform : n8n

- **n8n** est une plateforme opensource d'**automatisation de workflows** qui permet de connecter différentes applications, API et services entre eux, sans avoir à développer toute la logique manuellement.
- n8n signifie souvent "**NodemationN**".
- Solution low-code / no-code, avec possibilité d'ajouter du code si besoin.
- **Disponible en cloud ou en auto-hébergèrent.**
- Avec n8n, on peut par exemple :
 - récupérer des données depuis un formulaire ou un CRM,
 - les transformer,
 - envoyer un email ou une notification,
 - mettre à jour une base de données,
 - déclencher une action dans Slack, Google Sheets, Notion, WhatsApp, etc.
- n8n peut servir à construire des **agents IA opérationnels**, surtout pour des cas d'usage métier. Mais **n8n n'est pas, à lui seul, un framework agentique avancé** au même niveau que des outils spécialisés comme **LangGraph, CrewAI ou AutoGen**.

<https://www.youtube.com/watch?v=E0OZPy5fP8E>



Qu'est-ce que le Tool Calling ?

Définition · le LLM seul est un cerveau sans main — les outils sont ses doigts

DÉFINITION

Tool Calling

Mécanisme par lequel un **LLM** demande à un framework d'**exécuter une fonction externe** (API, calcul, base de données, recherche...) pour obtenir une information ou accomplir une action qu'il ne peut pas faire seul.

Un LLM sans outils = un cerveau isolé du monde réel.

ANALOGIE

LLM = cerveau

Raisonne -> Comprend -> Décide

+

outils

Pourquoi a-t-on besoin d'outils ?

Ce qu'un LLM seul ne peut pas faire ...

Info récente

Un LLM n'a pas Internet en temps réel : météo, cours boursier, actualités du jour.

Données privées

Il ne connaît pas votre base clients, vos emails, vos documents internes.

Calculs exacts

Il se trompe sur les grands nombres, les dates précises, les conversions d'unités.

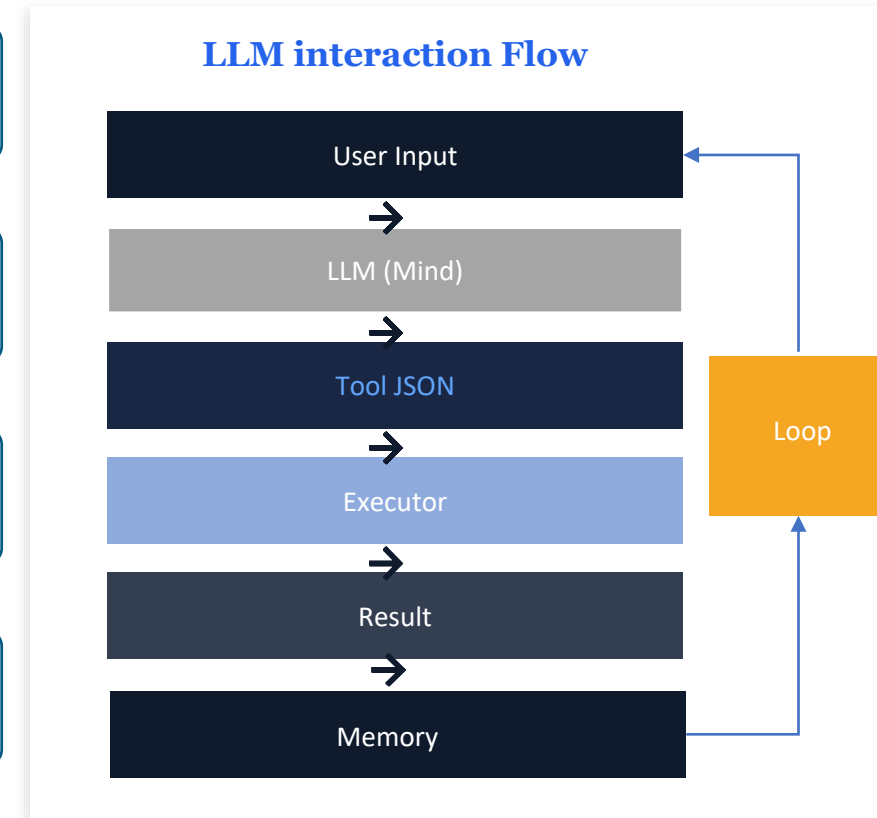
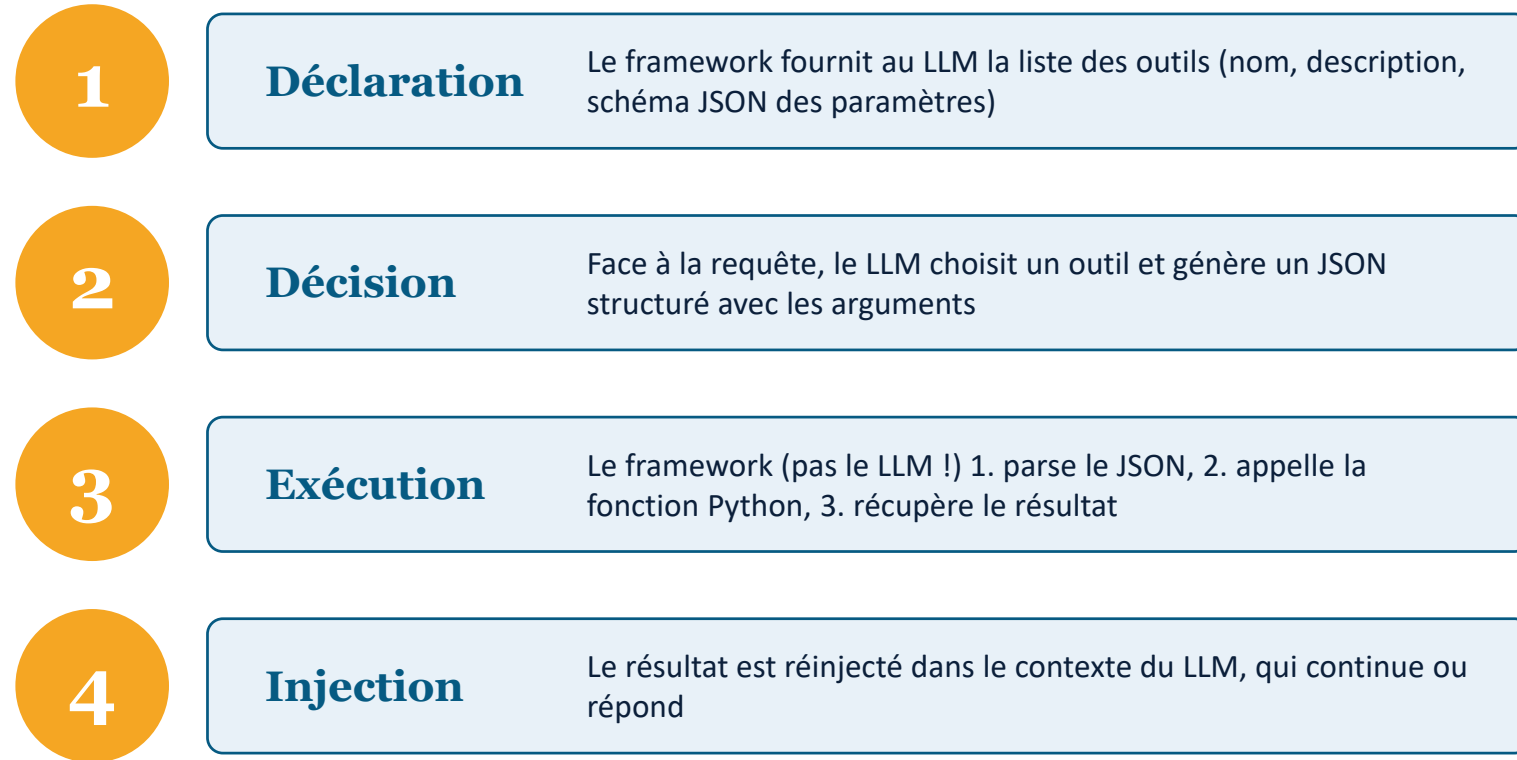
Agir

Envoyer un mail, créer une tâche, appeler une API : un LLM seul ne fait qu'écrire du texte.

→ *Le LLM raisonne, mais il lui faut des outils pour percevoir et agir sur le monde.*

Le cycle du Tool Calling — 4 étapes

Le **LLM** ne fait JAMAIS l'appel — il émet du **JSON**, le **framework** exécute



🔄 le LLM peut enchaîner plusieurs appels avant de répondre à l'utilisateur

Anatomie d'un outil

3 composants obligatoires — le LLM ne voit jamais votre code, il voit une carte d'identité

NAME <i>identifiant unique</i>	DESCRIPTION <i>lue par le LLM</i>	FUNCTION <i>le code Python exécuté</i>
Ce que le LLM mentionne quand il veut appeler l'outil. Verbe + objet. <u>Exemple:</u> search_wikipedia, get_weather).	Phrase(s) qui expliquent QUOI fait l'outil et QUAND l'utiliser. C'est la seule chose que le LLM lit pour choisir.	Le corps de la fonction. Exécuté par le framework après que le LLM ait fourni les arguments en JSON.

NAME · DESCRIPTION · FUNCTION — les trois piliers de tout outil bien conçu

Ce que le LLM voit réellement

Votre code Python → converti automatiquement en JSON schema par le framework

Votre code Python

```
@tool
def get_weather(city: str) -> str:
    """Donne la météo d'une ville.

    À utiliser quand l'utilisateur
    demande la température.

    Args:
        city: Nom de la ville
              (ex: Casablanca)
    """
    return api.fetch(city)
```



le framework
génère

Ce que le LLM reçoit (JSON)

```
{
  "name": "get_weather",
  "description": "Donne la météo
                  d'une ville. À utiliser
                  quand l'utilisateur...",
  "parameters": {
    "city": {
      "type": "string",
      "description": "Nom...",
      "required": true
    }
  }
}
```

nom fonction → name · docstring → description · type hints → types

Le décorateur `@tool` — définition

Une seule ligne au-dessus de votre fonction Python la transforme en outil utilisable par un LLM



Qu'est-ce qu'un décorateur ?

En Python, un décorateur est **une fonction** qui modifie le **comportement d'une autre fonction**, sans toucher à son code. On le place sur la ligne juste au-dessus, avec le symbole `@`.

Ce que fait `@tool` :

- Lit la signature de la fonction (types des arguments),
- Lit la docstring (description),
- Génère automatiquement le JSON schema,
- Enregistre l'outil pour qu'il soit callable par l'agent.

Syntaxe minimale

```
from langchain.tools import tool

@tool
def ma_fonction(x: str) -> str:
    """Description lue par le LLM."""
    return "résultat"
```

Une ligne · zéro configuration · tout est inféré

Bien décrire un outil — règles d'or

Docstring mal écrite = outil ignoré par le LLM, ou appelé avec de mauvais arguments

À ÉVITER

```
@tool
def search(q: str):
    """Recherche."""
    return api.find(q)
```

- ✗ **Nom vague** : search — chercher QUOI ?
- ✗ **Description d'1 mot** : n'explique ni **quoi** ni **quand**
- ✗ **Paramètre non documenté** : aucun exemple
- ✗ **Pas de section Args**

→ Le LLM hésitera ou enverra des arguments incorrects

À FAIRE

```
@tool
def search_wikipedia(query: str):
    """Recherche factuelle sur Wikipedia.
    À utiliser pour questions historiques,
    géographiques ou biographiques.
    Args:
        query: terme à chercher
               (ex: "PIB Maroc")"""
```

- ✓ **Nom explicite** : verbe + objet
- ✓ **Description = QUOI + QUAND**
- ✓ **Chaque paramètre typé + exemple**
- ✓ **Format Args reconnu par LangChain**

→ Le LLM choisit cet outil avec confiance et du 1er coup

Écrivez la **docstring** comme un **contrat** : QUOI fait l'outil · QUAND l'utiliser · QUELS arguments + exemples

Exemple complet — @tool en Python

Un outil Wikipedia, du décorateur à l'appel par un agent · 95% des cas d'usage

```
tool_wikipedia.py
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def search_wikipedia(query: str) -> str:
    """Recherche des informations sur Wikipedia.
    À utiliser pour des questions factuelles,
    historiques ou géographiques.

    Args:
        query: Le terme à chercher (ex: "Maroc")
    """
    result = wiki_api.search(query)
    return result

# Utilisation dans un agent
agent = create_agent(llm, tools=[search_wikipedia])
agent.invoke("PIB du Maroc en 2023 ?")
```

AVANTAGES

- Ultra concis (1 ligne)
- Type hints → schéma auto
- Docstring = description
- Idéal pour prototypage

LIMITES

- Peu de contrôle fin
- Pas de validation avancée
- Args simples uniquement
- Pour > passer à Pydantic / StructuredTool

Tool Calling & @tool en 5 idées clés

- 1 Le LLM n'exécute JAMAIS le code — il émet un JSON, le framework exécute la fonction Python.
- 2 Un outil = 3 composants : NAME (id unique), DESCRIPTION (lue par le LLM), FUNCTION (code exécuté).
- 3 Le LLM ne voit pas votre code : il voit un JSON schema généré depuis votre signature + docstring.
- 4 @tool transforme automatiquement une fonction Python en outil — une ligne, zéro configuration.
- 5 La docstring est un contrat : QUOI fait l'outil, QUAND l'utiliser, QUELS arguments avec exemples.

Frameworks & à retenir

Frameworks populaires

LangChain

Écosystème complet, chaînes & agents

LlamaIndex

Spécialisé RAG & data

CrewAI

Agents multi-rôles collaboratifs

AutoGen

Conversations multi-agents (MS)

À RETENIR

- ✓ Agent = Mind + Body + **Memory** + Loop
- ✓ Le LLM n'est qu'UN Mind parmi 6 (règles, recherche, ML, RL, LLM, hybride)
- ✓ La mémoire (court/long terme, épisodique, sémantique) distingue un vrai agent d'un chatbot
- ✓ ReAct = pattern de contrôle (Thought → Action → Observation) — pas un apprentissage
- ✓ Tool calling : le LLM émet du JSON, le framework exécute

Exemple concret

Comment l'agent décide — Relation Mind – Body Memory – Tool - ReAct

Je suis stressé pour mon examen.

- Mind ?
- Body (Action) ?
- Memory ?
- Tool (API/Fct) ?

Mind: Thought:

L'utilisateur est stressé à cause d'un examen.
Je dois proposer une aide concrète et personnalisée.

Action + Tool:

`get_study_plan(exam_date, user_profile)`

Observation (à stocker dans Memory):

Plan généré avec 3 sessions + pauses + révision finale.

Mind: Thought:

Je dois aussi réduire le stress immédiatement.

Action + Tool:

`launch_breathing_exercise()`

Observation (à stocker dans Memory):

Exercice de respiration lancé (2 minutes).

ReAct

- Thought:
- Action:
- Observation:

Question

Un agent sans mémoire est-il un agent ?

Answer:

Je comprends que tu es stressé.
Voici un plan de révision adapté + un exercice rapide pour te détendre.

« **Human-in-the-loop** » ou le mythe de l'**autonomie**.

Exécution d'**actions critiques**.

Tool calling ≠ autonomie

L0 → parler
L1 → agir
L2 → réfléchir
L3 → planifier
L4 → collaborer

Niveau d'autonomie	Description
L0	Réactif (chatbot)
L1	Tool calling
L2	ReAct
L3	Planification multi-étapes
L4	Multi-agents

Réaction
Action
Raisonnement
Planification
Collaboration

Méthode 2 — La classe Tool(...)

Instanciation manuelle · pour wrapper une fonction existante sans la modifier

```
from langchain.tools import Tool

# Fonction existante non décorée
def get_weather(city: str) -> str:
    return weather_api.fetch(city)

# On l'enveloppe dans un Tool
weather_tool = Tool(
    name="get_weather",
    func=get_weather,
    description="Donne la météo actuelle  
d'une ville donnée.  
Input: nom de ville."
)

agent = create_agent(llm, tools=[weather_tool])
```

✓ AVANTAGES

- Fonction intouchée
- Description personnalisée
- Utile pour libs tierces
- Nom ≠ nom de fonction

✗ LIMITES

- Un seul argument string
- Pas d'args multiples
- Plus verbeux que @tool

→ Utile quand on ne veut/peut pas modifier le code source de la fonction

Méthode 3 — StructuredTool (multi-arguments)

Quand l'outil prend PLUSIEURS paramètres structurés (pas juste une string)

```
from langchain.tools import StructuredTool

def convert_currency(
    amount: float,
    from_cur: str,
    to_cur: str
) -> float:
    rate = fx_api.get(from_cur, to_cur)
    return amount * rate

# from_function gère les args multiples
fx_tool = StructuredTool.from_function(
    func=convert_currency,
    name="convert_currency",
    description="Convertit entre devises"
)
```

✓ AVANTAGES

- N arguments typés
- Types complexes OK
- Validation via types
- Compromis flexibilité

✗ LIMITES

- Plus verbeux
- Validation basique
- Pas de règles métier

→ Choix par défaut dès qu'un outil a > 1 argument

Méthode 4 — Schéma Pydantic (validation stricte)

Pour la production : validation, contraintes, descriptions par champ

```
from pydantic import BaseModel, Field
from langchain.tools import StructuredTool

class SearchInput(BaseModel):
    query: str = Field(
        ..., description="Terme à chercher",
        min_length=3, max_length=200
    )
    lang: str = Field("fr", regex="^(fr|en|ar)$")

search_tool = StructuredTool.from_function(
    func=do_search,
    name="search",
    description="Recherche documentaire",
    args_schema=SearchInput # ← validation
)
```

✓ AVANTAGES

- Validation stricte
- Contraintes (min/max/regex)
- Description par champ
- Production-ready
- Erreurs explicites

✗ LIMITES

- Setup plus lourd
- Courbe d'apprentissage
- Overkill pour prototypes

→ Obligatoire dès qu'un outil touche à la production ou à des données sensibles

Quelle méthode choisir ?

Comparatif des 4 façons de définir un outil en LangChain

Méthode	Cas d'usage	Arguments	Validation	Verbosité	Quand l'utiliser
@tool	Prototypage rapide	1+ args simples	Types Python	Minimale	Toujours en 1er choix
Tool(...)	Wrapper fonction existante	1 string	Aucune	Moyenne	Code legacy / libs tierces
StructuredTool	Args multiples typés	N args typés	Via type hints	Moyenne	> 1 argument structuré
Pydantic Schema	Production / API publique	N args + contraintes	Stricte (regex, min/max)	Élevée	Données sensibles / prod

💡 Règle d'or : commencer par @tool → passer à StructuredTool si plusieurs args → Pydantic si validation critique